Computers and Chemical Engineering 115 (2018) 185–197

Contents lists available at ScienceDirect

Computers and Chemical Engineering

journal homepage: www.elsevier.com/locate/compchemeng

Deep convolutional neural network model based chemical process fault diagnosis

Hao Wu[a], Jinsong Zhao[a,b,*]

[a] State Key Laboratory of Chemical Engineering, Department of Chemical Engineering, Tsinghua University, Beijing, China
[b] Beijing Key Laboratory of Industrial Big Data System and Application, Tsinghua University, Beijing, China

ARTICLE INFO

Article history:
Received 17 November 2017
Revised 4 April 2018
Accepted 9 April 2018
Available online 11 April 2018

Keywords:
Fault diagnosis
Deep convolutional neural network
Alarm management
Tennessee Eastman process

ABSTRACT

Numerous accidents in chemical processes have caused emergency shutdowns, property losses, casualties and/or environmental disruptions in the chemical process industry. Fault detection and diagnosis (FDD) can help operators timely detect and diagnose abnormal situations, and take right actions to avoid adverse consequences. However, FDD is still far from widely practical applications. Over the past few years, deep convolutional neural network (DCNN) has shown excellent performance on machine-learning tasks. In this paper, a fault diagnosis method based on a DCNN model consisting of convolutional layers, pooling layers, dropout, fully connected layers is proposed for chemical process fault diagnosis. The benchmark Tennessee Eastman (TE) process is utilized to verify the outstanding performance of the fault diagnosis method.

© 2018 Elsevier Ltd. All rights reserved.

1. Introduction

With the wide application of distributed control systems (DCS) during the past three decades, chemical processes have become more and more automated. However, there have been many tragic chemical process accidents, which have resulted in fatalities as well as asset, environmental damages. It seems that we have not tackled the process safety issues head on. One critical protection layer is still missing. This layer currently is relying upon operators who need to be timely aware of abnormal situations and make corrective decisions. However, it requires safety intelligence of operators. Different operators may have different levels of safety intelligence. For an operator who is lack of enough safety intelligence, it is almost impossible for him/her to play this critical role. Therefore, there has been an industrial need to develop an intelligent fault detection and diagnosis (FDD) system to assist operators in handling abnormal situations.

Various FDD methods have been proposed in literature so far. Generally, the FDD methods can be classified into three categories: knowledge based, model based and data based methods. The process model-based methods can be further classified into qualitative model based and quantitative model based methods (Eslamloueyan et al., 2003; Venkatasubramanian et al., 2003a, 2003b, 2003c). Due to the insurmountable drawbacks of knowledge based and model based methods, data based methods usually stand out among the three categories, especially in the era of internet of things.

Data based FDD methods can be classified into statistical methods, shallow learning methods and deep learning methods. Statistical methods include principle component analysis (PCA) (Wise et al., 1990; Russell et al., 2000; Cho et al., 2005; Rato et al., 2016), independent component analysis (ICA) (Kano et al., 2003; Lee et al., 2007; Hsu et al., 2010; Ge et al., 2012; Fan and Wang, 2014), partial least squares (PLS) (MacGregor et al., 1994; Plovoso and Kosanovich, 1994; Zhang and Hu, 2011), fisher discriminant analysis (FDA) (Chiang et al., 2000; Zhu and Song, 2011), qualitative trend analysis (QTA) (Maurya et al., 2005, 2007, 2010) and their derivative methods. Based on the industrial benchmark of Tennessee Eastman (TE) process, a comparison study on these statistical methods was conducted for process monitoring and fault diagnosis (Yin et al., 2012).

Shallow learning methods include support vector machine (SVM) (Chiang et al., 2004; Kulkarni et al., 2005; Zhang, 2008; Mahadevan and Shah, 2009; Yélamos et al., 2009;), artificial immune system (AIS) (Dai and Zhao, 2011; Ghosh and Srinivasan, 2011; Shu and Zhao, 2016), k-nearest neighbor (KNN) (He and Wang, 2007), Gaussian mixture model (GMM) (Choi et al., 2004; Yu and Qin, 2008) and artificial neural network (ANN) (Venkatasubramanian and Chan, 1989; Watanabe et al., 1989; Fan et al., 1993). Shallow learning methods have been successfully utilized to treat fault diagnosis as a classification problem. Furthermore, some neural networks derived from ANN were developed for FDD, such as hierarchical ANN (HANN) (Watanabe et al., 1994; Eslamloueyan et al.,

* Corresponding author at: State Key Laboratory of Chemical Engineering, Department of Chemical Engineering, Tsinghua University, Beijing, China.
E-mail address: jinsongzhao@tsinghua.edu.cn (J. Zhao).

https://doi.org/10.1016/j.compchemeng.2018.04.009
0098-1354/© 2018 Elsevier Ltd. All rights reserved.

2003), Duty-Oriented HANN (DOHANN) (Eslamloueyan, 2011) and supervised local multilayer perceptron (SLMLP) (Ayubi and Yazdanpanah, 2015).

Despite of the advantages of the above two categories of data driven methods, FDD is still far from widely practical applications due to two major obstacles. One is that these two categories of FDD methods often require considerable amount of domain expertise to determine the fault features in both spatial and temporal domains. The other one is that the fault diagnosis rate is still not high enough.

Over the past few years, deep learning has become an outstanding technology and has shown better performance than the aforementioned methods in many fields. A deep learning architecture is a multilayer stack of several simple but non-linear layers, such as restricted Boltzmann machines (RBM), convolutional and pooling layers. The difference between "shallow" and "deep" is that, deep learning emphasizes the importance of feature extraction layer by layer. Each layer can transform the input representation at a low level into a representation at a higher and more abstract level (LeCun et al., 2015). With the composition of enough transformations, complex functions can be learned and the output representation of the last layer is easier for pattern recognition tasks. With the rapid development of deep learning, some deep learning based methods have been proposed for chemical process fault diagnosis. A hierarchical deep neural network (HDNN) was proposed for diagnosing the faults on the TE process (Xie and Li, 2015). The average correct classification rate reached 80.5% (except fault 03, 09, 15), which was higher than DOHANN. Lv et al. utilized stacked sparse auto encoder neural networks and a softmax classifier for FDD (Lv et al., 2016). In the latest study, an extensible deep belief network (DBN) based fault diagnosis model was proposed by the research group of the corresponding author (Zhang and Zhao, 2017). Features of fault data in spatial and temporal domains were extracted by DBN sub-networks, then a global back-propagation network was used for fault classification. Even though the average fault diagnosis rate for all the 20 fault types in the TE process reached 82.1%, an all-time high record, it is still far from real applications. Therefore, further researches are still needed.

The above three deep learning based methods are all based on DBN, which is a stack of RBMs. DBN was first proposed in 2006 (Hinton and Salakhutdinov, 2006a; Hinton et al., 2006b), which is widely regarded as the start of deep learning. However, in the famous ImageNet Large Scale Visual Recognition Challenge (ILSVRC) competition in 2012, AlexNet (Krizhevsky et al., 2012), the first model that used a deep convolutional neural network (DCNN) won the champion. Since then, DCNN has brought about a revolution in computer vision and pattern recognition. Now DCNN is the dominant approach for almost all recognition and detection tasks (LeCun et al., 2015). However, DCNN model based fault diagnosis method for chemical processes has been paid much less attention.

The basic architectures of typical convolutional neural networks mainly contain two types of layers for feature extraction: convolutional layers and pooling layers. In addition, other layers such as dropout and fully connected (FC) layers, are also significant for DCNN. Compared with DBN, DCNN has three advantages: the first one is that convolutional and pooling layers are local-connected with filters, which can help extract local patterns or features better; the second one is that DCNN with similarly sized layers has fewer parameters and requires less computation time; the third one is that "overfitting" can be avoided by using dropout and pooling layers.

This paper presents a DCNN model based fault diagnosis method for complex chemical processes. The rest of the paper is organized as follows: Section 2 introduces the basic theory of DCNN, including convolutional layers, pooling layers, dropout and FC layers. Section 3 presents the DCNN based chemical process fault diagnosis method in detail. Section 4 shows the experiment result of the fault diagnosis on the TE process. Finally, Section 5 summarizes this paper.

2. Deep convolutional neural network

Convolutional neural network (CNN) was first proposed in the late 1980 s, to process data that comes in the form of multiple arrays (LeCun et al., 1989). Since then, it has made great success in object detection and recognition in the computer-vision domain. Over the past few years, several outstanding DCNN architectures have been reported, including AlexNet (Krizhevsky et al., 2012), Network in Network (Lin et al., 2013), VGG (Simonyan and Zisserman, 2014), GoogLeNet (Szegedy et al., al.,2014), ResNet (He et al., 2016), etc. The general function of DCNN includes feature extraction and classification. For feature extraction, convolutional layers and pooling layers are stacked to transform the raw data into a representation at a higher level. Then fully connected layers are utilized to classify the transformed representation into certain class. Labeled data are required at the training phase which is based on the backpropagation procedure. At the beginning of the training phase, data samples are divided into mini-batches by providing a parameter named "batch size". The basic method for training neural networks is stochastic gradient descent (SGD). The idea of SGD is to update the weights and biases of neurons after each batch computation, the aim of which is to minimize the training error. The training of the multilayer architecture is a supervised learning process.

2.1. Convolutional layer

In a convolutional layer, the representation of the output is composed of feature maps, within which each unit is connected to a local patch in the input feature map through a filter composed of a set of weights. All units in an output feature map share the same filter (shared weights). Different feature maps in a layer use different filters. A typical convolutional layer is shown in Fig. 1. There are two reasons for using convolutional layers. First, in array data such as process data, local groups of values are often highly correlated, and local patterns can be formed to make pattern detection or recognition easier. Second, the distinctive local patterns can appear anywhere in the input feature maps, hence units at different locations sharing the same weights can help detect the same patterns regardless of their locations (LeCun et al., 2015).

In a convolutional layer, assume that there are M feature maps as the input and N filters. Generally, we can use Eq. (1) to calculate the output feature maps of the lth layer (Bouvrie, 2006):

$$x_j^l = f\left(\sum_{i=1,\ldots,M} x_i^{l-1} * k_{ij}^l + b_j^l\right), \quad j = 1, \ldots, N, \tag{1}$$

where k_{ij}^l represents the kernel of the jth filter connected to the ith input map, x_i^{l-1} represents the ith input map and x_j^l represents the jth output map, b_j^l represents the bias corresponding to jth filter, f represents the activation function, and $*$ represents the convolutional operation. In this way, we can obtain N feature maps as the output. Assuming that the kernel size is $s \times s$, we can use Eq. (2) to compute the number of all the parameters of a convolutional layer:

$$P = N \times \ (s \times s \times M + 1) \tag{2}$$

The convolutional operation is shown in Fig. 2, where the size of the input map is 4×4 ,the kernel size is 2×2, and the stride is 1. After the convolutional operation and the addition of the corresponding bias, an activation function is applied for computing the output feature maps. The common activation functions for

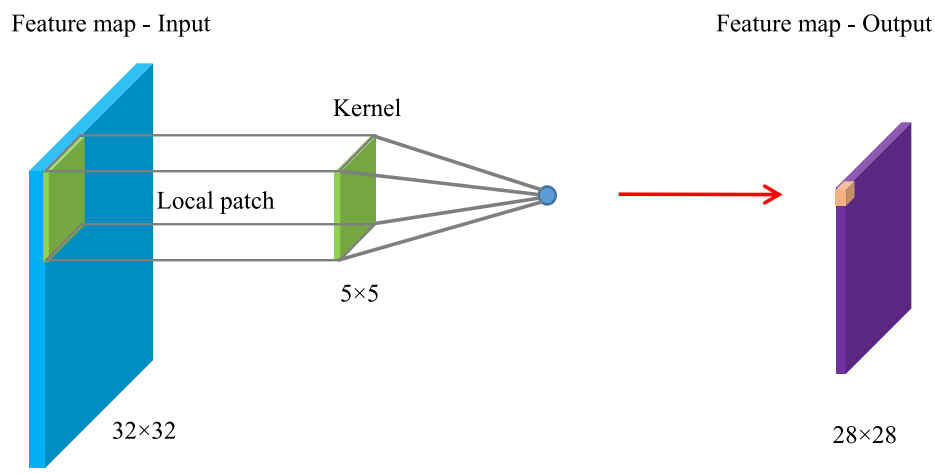


Fig. 1. Convolutional layer.

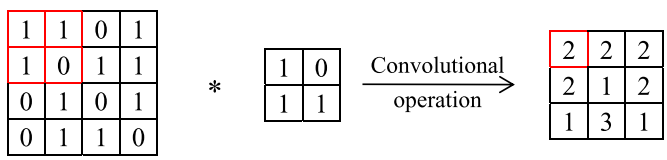


Fig. 2. Convolutional operation.

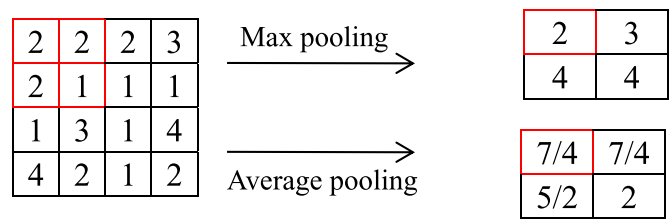


Fig. 3. Max pooling and average pooling operations.

neural networks include logistic function, hyperbolic tangent function and rectified linear unit (ReLU) function as shown in Eq. (3)–(5):

$$f(x) = \left(1 + e^{-x}\right)^{-1} \tag{3}$$

$$f(x) = \tanh(x) \tag{4}$$

$$f(x) = \max(0, x) \tag{5}$$

However, it has been proved that DCNN with ReLUs can be trained several times faster than their equivalents with the other functions (Krizhevsky et al., 2012). Since then, ReLU has become the first choice for designing a DCNN architecture.

2.2. Pooling layer

A pooling layer is also known as a sub-sampling layer which follows a convolutional layer and produces down-sampled versions of the input feature maps. The objective of a pooling layer is to merge similar local features into one. There are three advantages for the use of pooling layers. Firstly, because the relative positions of the features forming a local pattern may vary slightly, it is more reliable for detection to merge similar features in local positions (LeCun et al., 2015). Secondly, pooling layers usually do not have parameters and can reduce the dimension of the feature representation. The use of pooling layers can also greatly reduce the computation time and the parameters of the whole network. Thirdly, pooling layers are beneficial for preventing the "overfitting" (see Section 2.3).

There are two types of pooling operations: max pooling and average pooling. A max pooling unit computes the maximum of a local patch of units in a feature map, and an average pooling unit computes the average. The computation procedure in a pooling layer is similar with that in a convolutional layer. In a pooling layer, assuming that there are M feature maps as the input, there must be M output feature maps. Generally, we can use Eq. (6) to calculate the output feature maps of the lth layer (Bouvrie, 2006):

$$x_j^l = f\left(\beta_j^l down\left(x_j^{l-1}\right) + b_j^l\right), \quad j = 1, \ldots, M, \tag{6}$$

where x_j^{l-1} represents the jth input map and x_j^l represents the jth output map, b_j^l and β_j^l represent the additive bias and the multiplicative bias corresponding to the jth filter respectively, f represents the activation function, and *down* represents the sub-sampling function. Generally, β_j^l, b_j^l can be negligible and there is no parameter in pooling layers. In this way, we can obtain M feature maps as the output. The max pooling and average pooling operations are shown in Fig. 3, where the size of the input map is 4×4 ,the kernel size is 2×2, and the stride is 2.

2.3. Dropout

If a large neural network is trained on a small training dataset, it may have poor performance on the testing dataset. This phenomenon is called "overfitting". Generally, at the whole training phase, the training error will decrease with the number of iterations increasing. The testing error will decrease at the beginning of the training phase, however, will increase later. Fig. 4(a) shows the phenomenon of "overfitting". By using the dropout mechanism, the trend of the testing error will be similar to the trend of the training error (see Fig. 4(b)). It is very helpful for overcoming the "overfitting" problem.

Dropout avoids "overfitting" by randomly omitting some feature detectors in each iteration of the training stage (Hinton et al., 2012; Srivastava et al., 2014). Generally, in the dropout, we need to set the retaining probability p to a certain value (for example, $p = 0.5$) (see Fig. 5). It will set the output of each neuron to zero with the probability of 0.5 (Krizhevsky et al., 2012). This method can destroy complex co-adaptations of hidden neurons and prevent that a neuron is only useful in the context of several other specific neurons.

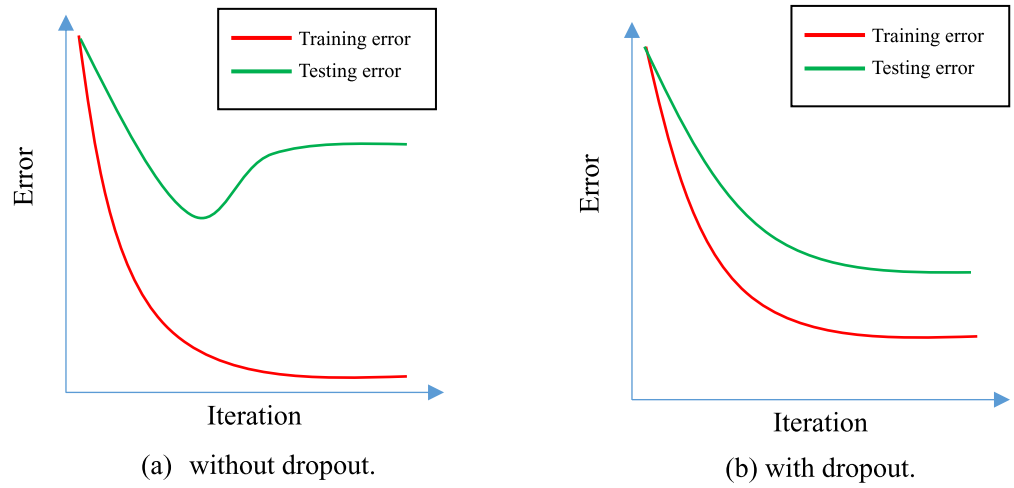


Fig. 4. The training error and the testing error.

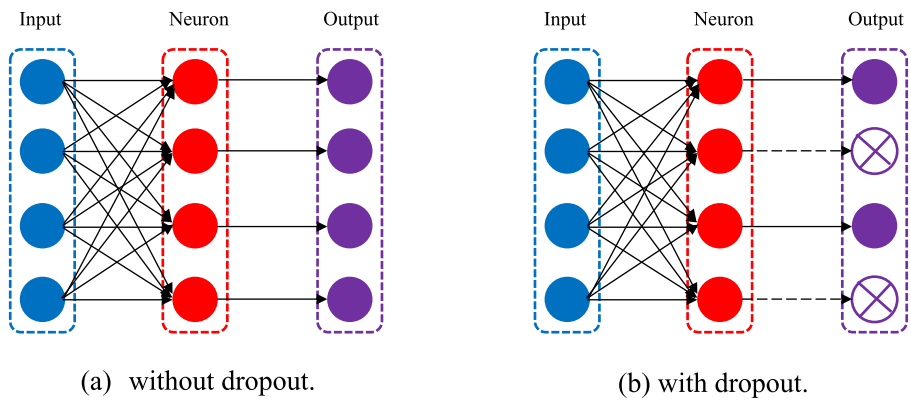


Fig. 5. Dropout neural net model.

The omitted half of the neurons will not be able to make contribution for the forward computation and the back propagation.

2.4. Fully connected layer

Convolutional layers and pooling layers constitute the feature extractor of the whole DCNN. Following the feature extractor, the objective of FC layers is to classify the features extracted from the raw data. FC layers are essentially backpropagation neural networks and their input must be one-dimensional vectors. Assume that the lengths of the input and output vectors are M and N respectively. Each value of the input vector is connected to each value of the output vector through one neuron (see Fig. 6). Then we can use Eq. (7) to calculate the output vector of the lth layer (Bouvrie, 2006):

$$x_j^l = f\left(\sum_{i=1,\ldots,M} x_i^{l-1} \times w_{ij}^l + b_j^l\right), \quad j = 1, \ldots, N, \tag{7}$$

where w_{ij}^l represents the weight of the jth output value connected to the ith input value, x_i^{l-1} represents the ith input value and x_j^l represents the jth output value, b_j^l represents the bias corresponding to the jth output value, and f represents the activation function. The computation for the number of all the parameters of a FC layer is as follows:

$$P = M \times N + N \tag{8}$$

A typical DCNN architecture contains several FC layers in the end. Due to the design of full connectivity, the parameters of FC layers are redundant, compared with other types of layers. The parameters of FC layers constitute more than 80% of parameters of the whole DCNN.

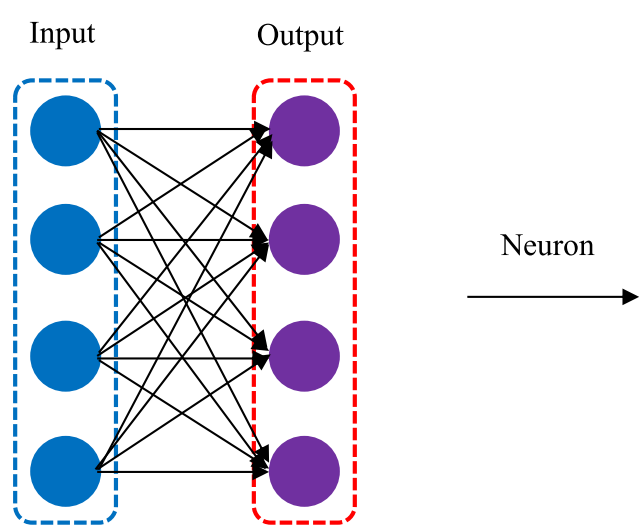


Fig. 6. Fully connected layer.

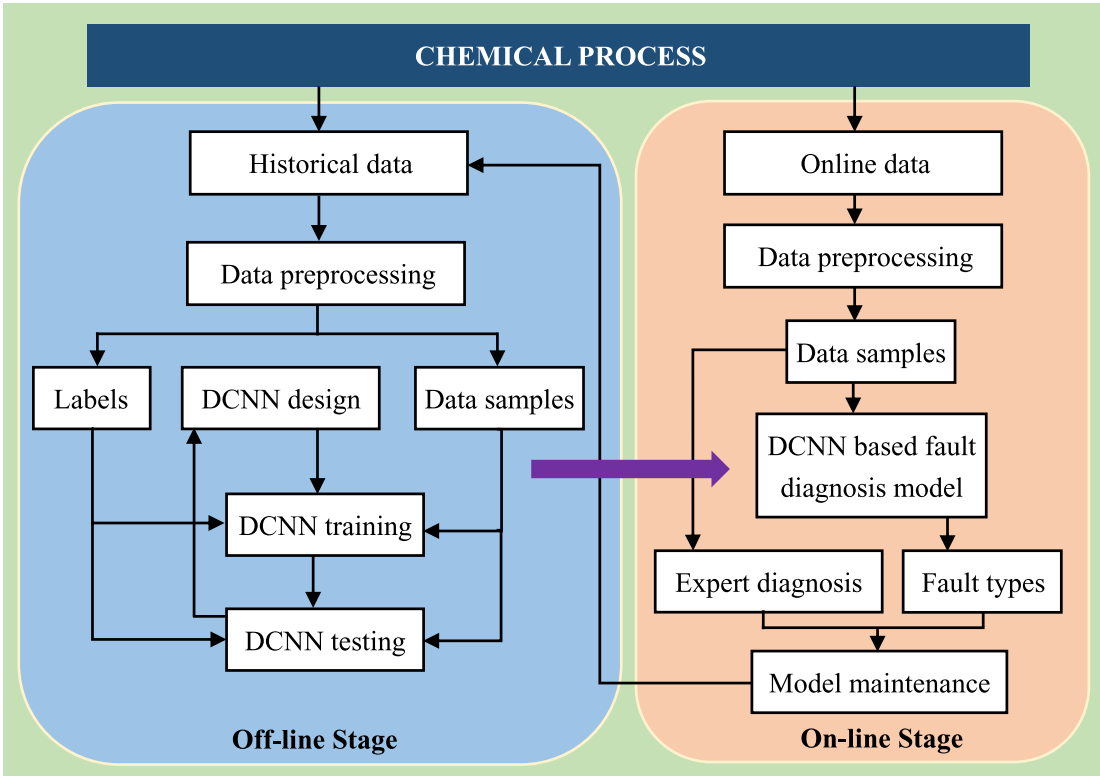


Fig. 7. The framework of the DCNN based fault diagnosis method.

3. DCNN based fault diagnosis method

Faults in chemical processes are essentially states that process variables deviate from their normal states. Data from different state deviations can be used to diagnose the fault types. Most of the published FDD methods only studied the features of fault data in the spatial domain, while the time-varying features in the temporal domain are relatively less studied. It has been proved that the time-varying features of fault data are also critical for distinguishing fault types (Maurya et al., 2005, 2007, 2010; Zhang and Zhao, 2017), which is also true even for human experts in performing various diagnosis tasks. Although DCNN is mainly used for images in computer vision, it can be used for spectrograms in speech recognition as well. DCNN extracts features from local patches, therefore, a simple data preprocessing is done in the proposed method to set the time-series of process variables of each equipment next to each other. In this way, there exists spatial relationship along the variable dimension. Similar with spectrograms including time and frequency domains, the data preprocessing also transforms the process data into two-dimensional matrices (temporal and spatial domains) with the size of $m \times n$, where m represents the length of a certain period of time (sample time length) and n represents the number of variables. For example, with $n = 50$, $m = 20$ (sampling period is set to 3 min), this means that the data of 50 variables from time t-1 h to time t is used as a sample matrix for diagnosing the status of time t.

The framework of the DCNN based fault diagnosis method is shown in Fig. 7. Its diagnosis procedures including offline and online stages are described as follows:

Offline stage:

Step 1: Historical data is collected and preprocessed from the chemical process.
Step 2: Through the data preprocessing, historical data is transformed into sample matrices with the size of $m \times n$ and labelled with their corresponding classes, including "normal" and their fault types (see Fig. 8(a)).
Step 3: The sample matrices including their corresponding labels are divided into the training set and the testing set.
Step 4: The DCNN model is designed for the chemical process (see Fig. 8(b)).
Step 5: The DCNN model is trained.
Step 6: The DCNN model is tested.
Step 7: The fault diagnosis result is outputted and visualized (see Fig. 8(c)).
Step 8: If the fault diagnosis rate in testing is satisfactory, the model will be used for online fault diagnosis; if unsatisfactory, the DCNN model needs to be redesigned (Step 4).

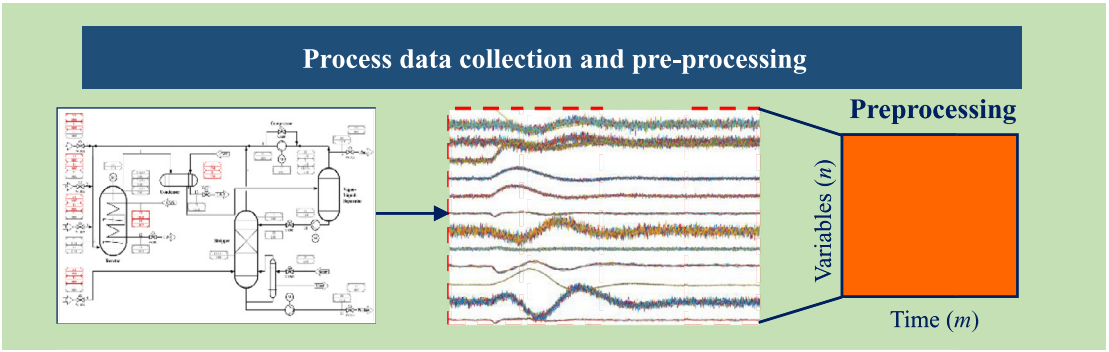
Online stage:

Step 1: Online data is collected and preprocessed from the chemical process.
Step 2: Through the same preprocessing, online data is transformed into sample matrices with the size of $m \times n$.
Step 3: Online sample matrices are input to the DCNN model. The model can give a predicted diagnosis result for each sample matrix. The diagnosis result is either "normal" or one specific fault type.
Step 4: If there is a discrepancy between the predicted diagnosis result and the judgment of the human experts, the DCNN model needs to be retrained with the new data.

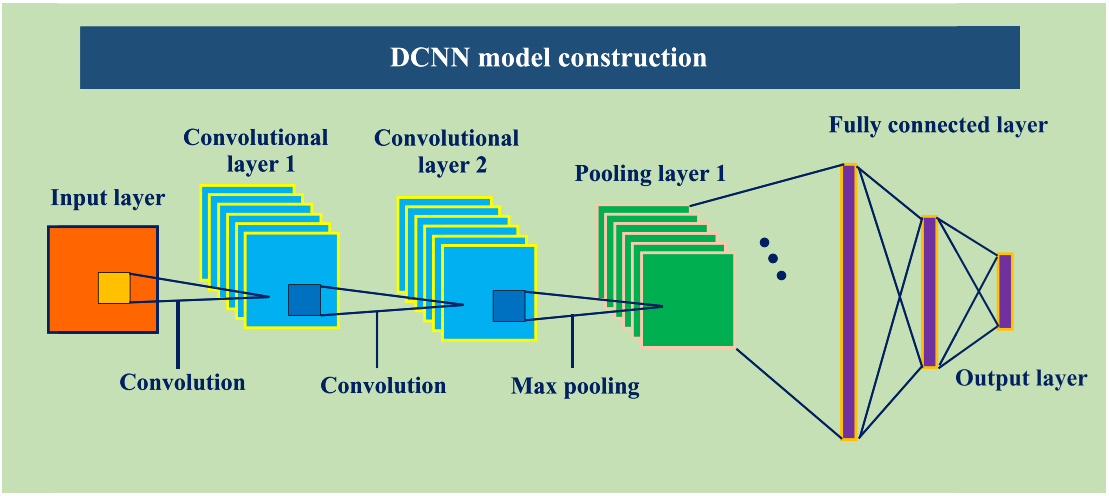
4. Experiment result

4.1. Tennessee Eastman process

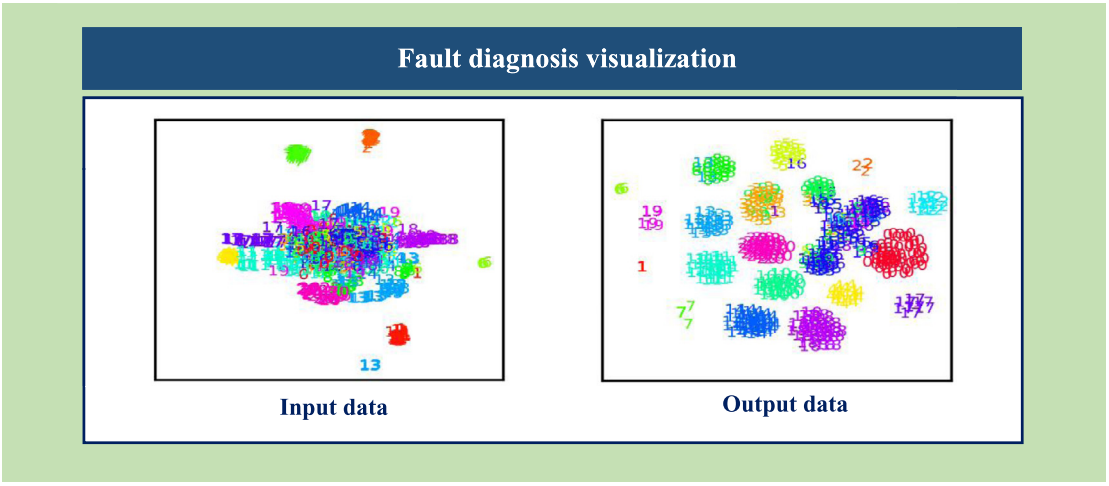
The benchmark Tennessee Eastman (TE) process (Downs and Vogel, 1993) is used for showing the advantages of the proposed DCNN model in this paper. The simulator is based on the revised



(a) Process data collection and preprocessing.



(b) DCNN model construction.



(c) Fault diagnosis visualization using t-SNE.

Fig. 8. The DCNN model for fault diagnosis.

version (see Fig. 9) which is available at http://depts.washington.edu/control/LARRY/TE/download.html (Bathelt et al., 2015). The process variables include 12 process manipulated variables, 22 continuous process measurements and 19 component analysis measurements. Even though there are 28 process disturbances (fault types) in the revised version, we only select IDV(1) – IDV(20) for comparison with other algorithms. Normal data and fault data of the TE process are collected from the simulations on MATLAB 2016a (see Table 1 and Table 2). The method of the simulations refers to Zhang's research (Zhang and Zhao, 2017). The sampling period is set to 3 min (20 samples/h). The simulator runs for 500 h in the normal state. 10,000 normal sample matrices are then collected and each contains one-hour data. In each simulation of the 20 faults, the simulator runs for 10 h in the normal state at the beginning. Then the corresponding fault disturbance is introduced and the simulator continue to run for 40 h. In this way, 40 h fault data (800 fault sample matrices) are collected in each simulation. It must be noted that the simulations of fault 06 shut down after

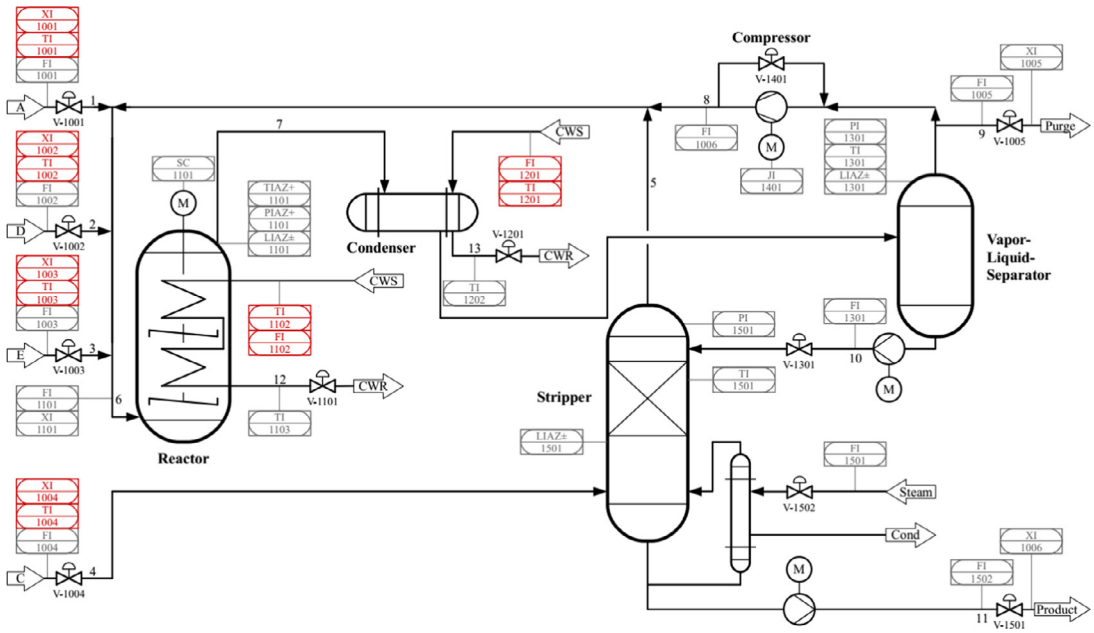


Fig. 9. P&ID of TE process (Bathelt et al., 2015).

Table 1
The fault sample matrices collected from the TE process simulator.

Status index	Training time length/h	Number of training sample matrices	Testing time length/h	Number of testing sample matrices
IDV 01-05 07-20	$40 \times 19 \times 8$	$800 \times 19 \times 8$	$40 \times 19 \times 2$	$800 \times 19 \times 2$
IDV 06	7×8	140×8	7×2	140×2
Fault	6136	122720	1534	30680

Table 2
The normal sample matrices collected from the TE process simulator.

Status index	Training time length/h	Number of training sample matrices	Testing time length/h	Number of testing sample matrices
Normal	400	8000	100	2000

7 h in the fault state, hence each simulation of fault 06 only has 7 h fault data. The simulation of each fault type repeats 10 times with 10 different initial states. We randomly select 80% normal sample matrices (8000) as training normal sample matrices and the remaining normal sample matrices (2000) as testing normal sample matrices. In the same way, for each fault, we randomly select 122,720 training fault sample matrices from eight simulations and 30,680 testing fault sample matrices from the other two simulations.

In order to extract the features of process data in both spatial and temporal domains, data samples are transformed into two-dimensional matrices with $m \times n$. In the TE process, XMV(5) (Compressor recycle valve), XMV(9) (Stripper steam valve) and XMV(12) (Agitator speed) are constant during the simulations. These three variables are excluded and therefore each sample matrix includes the information of the rest 50 variables for 1 h ($m = 20$, $n = 50$).

4.2. DCNN model for the TE process

It is a common issue that there is no scientific guidance for designing an optimal DCNN architecture. In order to find a proper model, we tried several DCNN models, the architectures of which are shown in Table 3. Then an architecture with the most outstanding fault diagnosis performance was selected from Table 3. Parameters of the selected architecture were studied through experiments. The parameters mainly contain the number of filters of each convolutional layer and the output length of the first FC layer.

Table 3
DCNN model candidates for fault diagnosis of the TE process.

Model	Architecture
Model 1	Conv(128)-Pool-FC(300)*-FC (21)
Model 2	Conv(128)-Conv(128)-Pool-FC(300)*-FC(21)
Model 3	Conv(128)-Conv(128)-Conv(128)-Pool-FC(300)*-FC(21)
Model 4	Conv(128)-Conv(128)-Conv(128)-Conv(128)-Pool-FC(300)*-FC(21)
Model 5	Conv(64)-Pool-Conv(128)-Pool-FC(300)*-FC(21)
Model 6	Conv(64)-Conv(64)-Pool-Conv(128)-Pool-FC(300)*-FC(21)
Model 7	Conv(64)-Conv(64)-Pool-Conv(128)-Pool(2×1)-FC(300)*-FC(21)
Model 8	Conv(64)-Conv(64)-Pool-Conv(128)-Pool(1×2)-FC(300)*-FC(21)
Model 9	Conv(64)-Pool-Conv(128)-Conv(128)-Pool-FC(300)*-FC(21)
Model 10	Conv(64)-Conv(64)-Pool-Conv(128)-Conv(128)-Pool-FC(300)*-FC(21)
Model 11	Conv(64)-Conv(64)-Conv(64)-Pool- -Conv(128)-Conv(128)-Pool(2×1)-FC(300)*-FC(21)
Model 12	Conv(64)-Conv(64)-Pool- -Conv(128)-Conv(128)-Conv(128)-Pool(2×1)-FC(300)*-FC(21)

FC*: Dropout ($p = 0.5$) is utilized for this FC layer.
Pool: The default of kernel size in pooling layers is 2×2 and all the strides are set to 2.

In the following, *Model 7* is explained as an example. The input size of one sample matrix is 20×50, where "20" represents sample time length and "50" represents the number of process variables. Here we use 3 convolutional layers, 2 max pooling layers and 2 FC layers. In the three convolutional layers, the kernel sizes are all set to 3×3 and the strides are set to 1. The first and second convolutional layers both contain 64 filters and the

Table 4
Confusion matrix for the *i*th class.

	Number of samples in the *i*th class (Predicted)	Number of samples in the other classes (Predicted)
Number of samples in the *i*th class (Actual)	*p*	*b*
Number of samples in the other classes (Actual)	*q*	*d*

third contains 128 filters. The two max pooling layers are behind the second and third convolutional layers. The kernel size is set to 2 × 2 in the first pooling layer and set to 2 × 1 in the second pooling layer. Here the output of one sample matrix is a three-dimensional array (3 × 21 × 128). As mentioned in Section 2.4, the input of FC layers must be a one-dimensional vector, hence we utilize a "Flatten" layer to reshape three-dimensional arrays into one-dimensional vectors with the size of 8064 (3 × 21 × 128). The output length of the first FC layer is set to 300 and "Dropout" is used for this layer. The last FC layer outputs the classes of sample matrices with a "softmax" function. The softmax function also named normalized exponential function, is a generalization of the logistic function that transforms a *K*-dimensional vector *Z* of arbitrary real values into a *K*-dimensional vector $\sigma(Z)$ of real values in the range of (0, 1) that add up to 1. The function is shown as:

$$\sigma(Z)_j = \frac{e^{Z_j}}{\sum_{k=1}^{K} e^{Z_k}}, \quad j = 1, 2, \ldots, K, \tag{9}$$

After the transformation of the softmax function, the model outputs a vector of length 21, each value of which represents the possibility of the corresponding class. For testing a sample matrix, the predicted diagnosis result is the class with the highest possibility.

4.3. Fault diagnosis result

4.3.1. FDR and FPR

After the data collection and the DCNN model construction are completed, the training and testing procedures of classification are implemented in a server computer. In order to show the experiment result of the fault diagnosis, define the confusion matrix, fault diagnosis rate (FDR) and false positive rate (FPR) for the *i*th class in Table 4, Eq. (10) and (11), respectively:

$$\mathrm{FDR} = \frac{p}{p+b} \tag{10}$$

$$\mathrm{FPR} = \frac{q}{q+d} \tag{11}$$

Here the batch size for training is set to 128 and the number of epochs is set to 50. "Batch size" means the number of sample matrices in one forward/backward pass of each iteration. "One Epoch" means all the training sample matrices are passed forward and backward through the network only once. The fault diagnosis testing is on the basis of one sample matrix each time. Each sample matrix contains the time-series data of the 50 variables from time *t*-1 h to time *t* to diagnose the status of time *t*. The testing average FDR and the training/testing time of the models in Table 3 are listed in Table 5. *Model* 3 has the highest testing average FDR (88.4%) and takes 56 s × 50 = 46.7 min for training. With a little decrease of the testing average FDR (88.2%), *Model* 7 only takes about half the training time of *Model* 3. In the following discussion, *Model* 7 is chosen as the best architecture. Then through a lot of experiments, the parameters of Model 7 are set to {conv(64), conv(64), conv(128), FC(300)}.

Fig. 10 illustrates the curves of the accuracy at the training and testing phases. The DCNN based fault diagnosis results are listed in Table 6. Among the 122,720 training sample matrices, the average FDR of training dataset is 98.6% and the corresponding average FPR is 0.1%. As for the testing dataset including 30,680 sample matrices, the average FDR reaches 88.2% and the corresponding average FPR is 0.5%. These results show that except "Fault 09" (D feed temperature in stream 2 – random variation), "Fault 15" (Condenser cooling water valve - sticking) and "Fault 16" (Unknown), the other fault types can be diagnosed with more than 91% FDRs by the DCNN model. The details of the testing result for fault diagnosis are illustrated in Fig. 11. It shows the confusion matrix of all the 21 classes. "Size" and "Color" both represent the value of the confusion matrix.

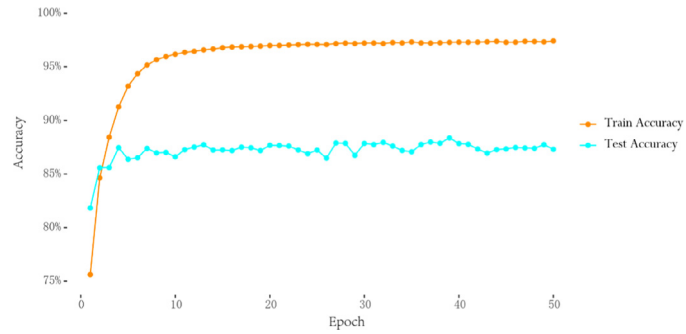


Fig. 10. The training accuracy and testing accuracy for the iteration process.

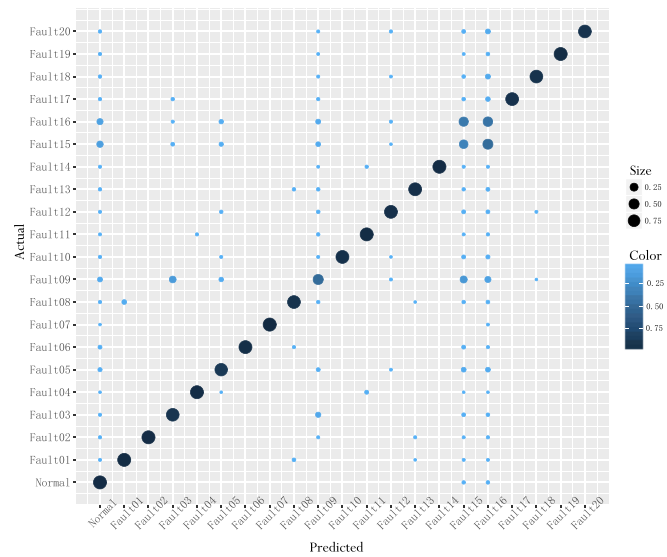


Fig. 11. The details of the testing result for fault diagnosis.

In the previous researches about the TE process fault diagnosis, the best reported results were achieved by the DBN based model (Zhang and Zhao, 2017). In order to show the performance of the proposed method, the results of the DBN based model and the DCNN based model are compared in Table 6. For the training dataset, the DBN based model cannot diagnose the three fault "Fault 09", "Fault 15" and "Fault 16" and the FDRs of "Fault 15" and "Fault 16" are even 0%. By contrast, the DCNN based model shows excellent performance on all the fault types in the training dataset, with 98.6% average FDR. For the testing dataset, the FDRs of "Fault 05", "Fault 08", "Fault 11", "Fault 12", "Fault 13", "Fault 14" are less

Table 5
The testing average FDR and the training and testing time.

Model	Testing average FDR (%)	Training time for one epoch (s)	Testing time for one sample matrix (ms)
Model 1	85.0	60	1.9
Model 2	87.9	57	1.8
Model 3	**88.4**	**56**	**1.8**
Model 4	88.0	56	1.8
Model 5	86.4	24	1.5
Model 6	87.5	24	1.5
Model 7	**88.2**	**30**	**1.5**
Model 8	87.5	31	1.5
Model 9	86.5	24	1.5
Model 10	87.5	24	1.5
Model 11	87.5	31	1.5
Model 12	87.8	27	1.5

Table 6
The comparison of fault diagnosis results between the DBN based model (Zhang and Zhao, 2017) and the DCNN based model.

Status index	FDR-Train (DBN)	FDR-Train (DCNN)	FPR-Train (DCNN)	FDR-Test (DBN)	FDR-Test (DCNN)	FPR-Test (DCNN)
Normal	-	0.916	0.001	-	0.978	0.015
Fault 01	1.00	0.998	0.000	1.00	0.986	0.003
Fault 02	1.00	0.996	0.000	0.99	0.985	0.000
Fault 03	0.99	0.996	0.000	0.95	0.917	0.008
Fault 04	0.98	0.999	0.000	0.98	0.976	0.000
Fault 05	0.90	0.998	0.000	0.86	0.915	0.006
Fault 06	1.00	0.998	0.000	1.00	0.975	0.000
Fault 07	1.00	0.999	0.000	1.00	0.999	0.000
Fault 08	0.96	0.985	0.001	0.78	0.922	0.001
Fault 09	0.655	0.973	0.000	0.57	0.584	0.020
Fault 10	0.975	0.977	0.000	0.98	0.964	0.000
Fault 11	0.975	0.995	0.000	0.87	0.984	0.001
Fault 12	0.855	0.992	0.000	0.85	0.956	0.001
Fault 13	0.965	0.978	0.001	0.88	0.957	0.000
Fault 14	0.96	0.998	0.000	0.87	0.987	0.000
Fault 15	0	0.997	0.001	0	0.28	0.028
Fault 16	0	0.912	0.009	0	0.442	0.038
Fault 17	1.00	0.988	0.004	1.00	0.945	0.000
Fault 18	1.00	0.970	0.002	0.98	0.939	0.001
Fault 19	0.97	0.996	0.000	0.93	0.986	0.000
Fault 20	0.987	0.971	0.001	0.93	0.933	0.000
Average*	**0.859**	**0.986**	**0.001**	**0.821**	**0.882**	**0.005**

*The average excludes the FDR of normal status because it is not available in the research of DBN based model.

than 88% in the DBN based model. By contrast, the proposed DCNN model achieves more than 91% FDRs of the six fault types. Additionally, the DCNN based model can partially diagnose “Fault 15" and “Fault 16", despite that their FDRs are still low. Overall, the average of testing FDRs through the DCNN based model is 88.2%, which is 6.1% higher than that of the DBN based model.

4.3.2. Hierarchical feature learning visualization

To facilitate the understanding of the feature learning process of the DCNN model, it is vital to learn about its hierarchical feature learning process. However, it is difficult for the output representation of each layer to visualize the diagnosis because the learned features are high-dimensional. To solve this problem, we adopt the t-distributed stochastic neighbor embedding (t-SNE) (Maaten and Hinton, 2008) method, which is usually used to visualize the hierarchical feature learning process of the DCNN model.

The t-SNE method is a variation of Stochastic Neighbor Embedding (SNE) (Hinton and Roweis, 2002), and is better at revealing the distribution of high-dimensional data. The t-SNE can embed high-dimensional features of each layer into a space of two or three dimensions, which can be visualized in a scatter map. With the 2D or 3D map corresponding to each layer, we can visualize the feature learning process easily. Through the experiment, we find that 3D maps are not suitable for visualization of the DCNN based fault diagnosis model. Therefore, the high-dimensional output features of each layer are embedded into 2D maps, which are then plotted in the subfigures in Fig. 12.

800 sample matrices of the 21 classes (one normal class and 20 fault classes) are randomly selected from the testing set for visualization. The size of the input data is $800 \times 20 \times 50$. Then the t-SNE method is used to transform these 20×50 sample matrices into 800 vectors of length 2. In Fig. 12(a), each point represents a sample matrix, plotted by the first value of each vector on the horizontal axis and the second value on the vertical axis. The points are marked with their actual class labels, “Normal” labelled with “0", “Fault 01" labelled with “1", etc. Additionally, in order to distinguish the clusters for viewing, different colors are used to represent their class labels as well. Similarly, the output of each layer is transformed by the t-SNE method into a vector of length 2 so that it can be visualized in Fig. 12(b)–(h). It should be noted that the output of the “Dropout” and “Flatten” layers are not visualized because these layers are useless for feature learning.

As illustrated in Fig. 12(a), the raw process data samples of all the classes are mixed up. Through three convolutional layers and two max pooling layers, we can find that the samples are gradually clustered by the class labels in the t-SNE maps (see Fig. 12(b)–(f)). Theoretically, clearer clusters mean better performance for classification. Finally, Fig. 12(g) and (h) are the t-SNE maps of the last two fully connected layers. The last t-SNE map illustrates the result of classification, and shows rather clear clustering of samples. These subfigures powerfully prove that the DCNN model is effective for the fault diagnosis task. Additionally, in Fig. 12(h), we can also find that the points with labels “9", “15" and “16" are mixed

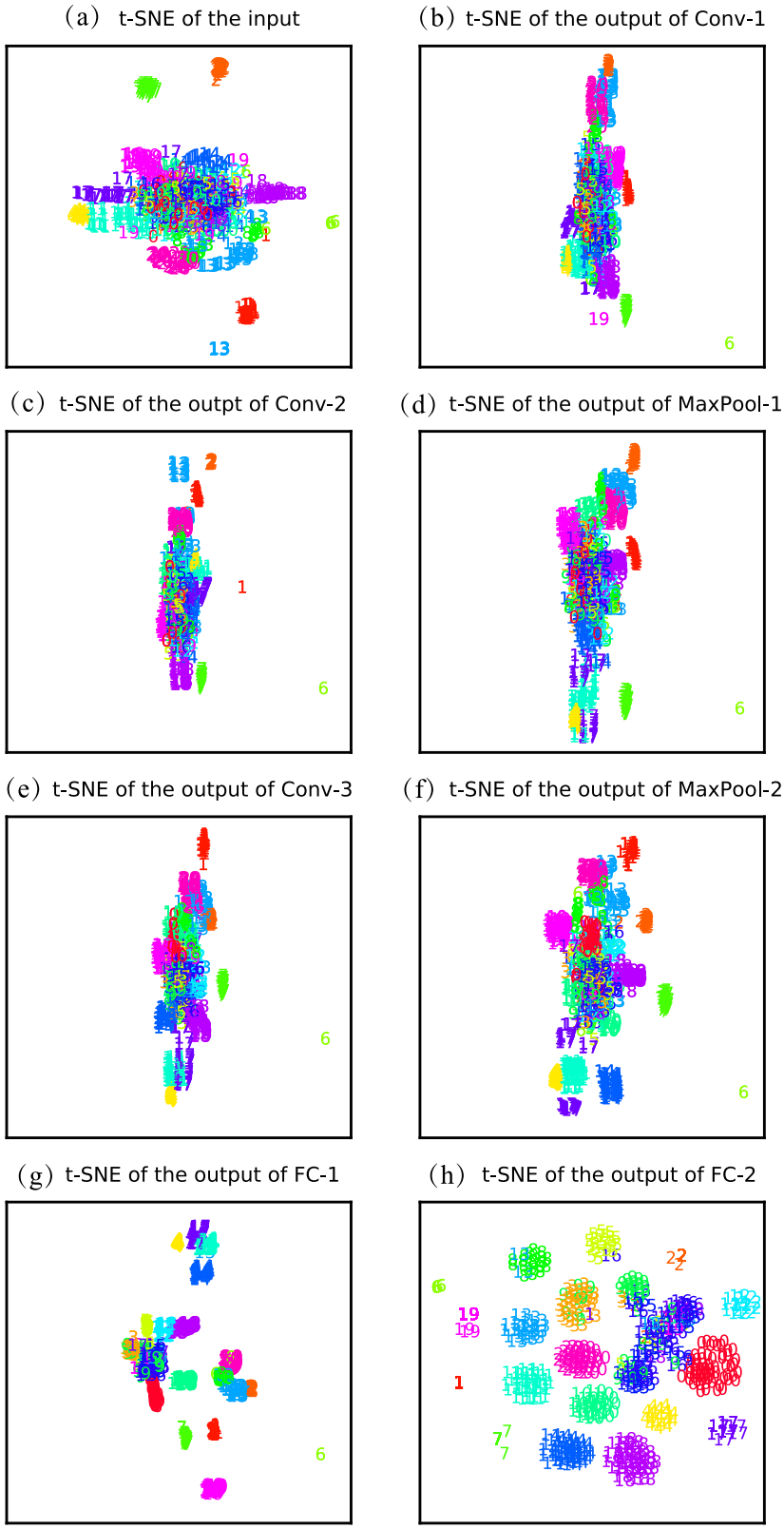


Fig. 12. DCNN model visualization using t-SNE.

up in the t-SNE map. It is well known that the FDRs of these three fault types in the previously reported researches were also quite low (Zhang and Zhao, 2017; Rato and Reis, 2017). Since the fluctuation ranges of the root cause variables of these three faults are unknown in the 50 variables of the TE simulator (Bathelt et al., 2015), it is hard to explain why their FDRs are low.

Model 7 is retrained by using the data of normal and the other 17 fault types (without "Fault 09", "Fault 15" and "Fault 16"). The result is listed in Table 7.

Table 7
The fault diagnosis result without "Fault 09", "Fault 15" and "Fault 16".

Status index	FDR(Test)	FPR(Test)	Status index	FDR(Test)	FPR(Test)
Normal	0.996	0.022	Fault 10	0.959	0.000
Fault 01	0.995	0.002	Fault 11	0.984	0.000
Fault 02	0.990	0.000	Fault 12	0.968	0.002
Fault 03	0.961	0.001	Fault 13	0.939	0.000
Fault 04	0.999	0.001	Fault 14	0.992	0.000
Fault 05	0.951	0.001	Fault 17	0.950	0.000
Fault 06	0.989	0.000	Fault 18	0.946	0.000
Fault 07	1.000	0.000	Fault 19	0.985	0.000
Fault 08	0.938	0.002	Fault 20	0.940	0.000
			Average	**0.970**	**0.001**

Table 8
The result of tuning the model output.

n_0	False alarm rate	Average FDR
0.5	0.004	0.970
0.4	0.003	0.969
0.3	0.002	0.969
0.2	0.002	0.968
0.1	0.002	0.967
0.005	0.001	0.961
0.004	0.0004	0.961

Table 9
The average FDR during different time periods after the fault introduction.

Time period	0~1h	1~2h	2~3h	3~4h	4~5h	5~6h	6~7h	7~8h
Average FDR	0.432	0.766	0.941	1.000	0.985	0.999	1.000	1.000

4.3.3. Model tuning

Table 7 shows that the FDR for normal operation is 99.6%. This means that 0.4% of normal data is misdiagnosed as fault data, which is usually regarded as "false alarm". If the false alarm rate is large, operators would be bombarded with lots of false alarms during normal operation. In order to minimize the false alarm rate, the output of the DCNN model can be tuned. After the softmax function, the model outputs a vector of length 18, $\{r_i | i = 0 \sim 8, 10 \sim 14, 17 \sim 20\}$, each value of which represents the possibility of the corresponding class. The diagnosis result (R) is the class with the highest possibility, $R = \text{argmax}_i(r_i)$. To reduce the normal data misdiagnosed as fault data, the following procedure is used for diagnosing the class of data:

set n_0;
if $r_0 > n_0$:
  return $R = 0$;
else :
  return $R = \arg\max_i(r_i)$

Table 8 lists the false alarm rates of the normal state and the average FDRs of all the fault types with several values of n_0. With the decrease of n_0, the false alarm rate decreases but the average FDR of faults decreases as well. This brings out the trade-off between the needs of false alarm rate and fault diagnosis rate.

4.3.4. Diagnostic performance

The fault development is a dynamic process. Therefore, it is necessary to explore the diagnostic performance of the proposed method as time progresses after the fault introduction. Table 9 lists the average FDR during different time periods after the fault introduction. It can be found that the average FDR increases as time progresses.

Furthermore, the diagnosis time is explored to show when the fault can be diagnosed correctly after the fault occurrence. In the aforementioned discussion, sampling period (A*) is set to 3 min, fault simulating time (B*) is set to 40 h, and sample time length (C*) is set to 20 ($20 \times 3\text{min} = 1\,\text{h}$). For comparison, the diagnosis time in different simulation conditions is listed in Table 10. If a fault is diagnosed consecutively for 5 (when A* = 3 min) evaluations, or 9 (when A* = 1 min) evaluations, or 30 (when A* = 15 s) evaluations, a diagnostic result is indicated. It is important to note that before the diagnosis time, one or several faults in the 17 fault types may be misdiagnosed. For example, in the condition of (A* = 3 min, B* = 40 h, C* = 20), 16 fault types can be correctly diagnosed when the first fault is detected. However, "Fault 13" will be misdiagnosed as "Fault 02" at 85 min after the fault occurrence. From Table 10, it can be found that as the sampling period decreases (from 3 min to 15 s), the faults can be correctly diagnosed early.

Table 10
The diagnosis time (min) in different simulation conditions.

A*	3min				1min				15s	
B*	40 h		20 h		20 h		10 h		20 h	
C*	20	10	20	10	20	10	20	10	20	10
Fault 01	28	36	30	31	16	18	18	18	15	13
Fault 02	39	50	60	47	17	19	17	15	17	18
Fault 03	35	26	34	26	14	18	19	17	9	7
Fault 04	15	34	16	38	9	9	9	10	8	8
Fault 05	33	12	28	13	10	9	10	9	26	22
Fault 06	18	25	23	27	11	12	11	12	9	8
Fault 07	15	15	15	15	9	9	9	9	7	7
Fault 08	94	89	97	92	44	38	13	44	81	71
Fault 10	82	57	91	61	31	39	40	39	24	52
Fault 11	45	45	56	43	12	17	16	16	19	18
Fault 12	52	33	66	44	13	18	27	20	13	9
Fault 13	100	82	107	101	131	105	77	86	72	62
Fault 14	24	24	27	23	15	16	15	17	12	12
Fault 17	124	123	123	115	27	34	35	42	43	41
Fault 18	45	56	107	96	36	72	43	28	19	20
Fault 19	64	53	59	51	17	18	17	18	17	15
Fault 20	146	163	134	154	79	60	66	29	44	62
Average	56	54	63	57	29	30	26	25	26	26

A*: sampling period; B*: fault simulating time; C*: sample time length.

5. Conclusion

In this paper, a DCNN model based chemical process fault diagnosis method is proposed. In order to extract the features in both spatial and temporal domains, a DCNN model is built with convolutional layers, pooling layers, dropout and FC layers. Raw process data is transformed into a $m \times n$ dimensional feature map, where m represents the length of a certain period of time (sample time length) and n represents the number of variables.

The experiment result shows that the proposed DCNN based fault diagnosis method has excellent performance on the benchmark TE process. The average FDR of all the 20 fault types reaches 88.2%, which is higher than other fault diagnosis methods published in literatures. Except three fault types ("Fault 09", "Fault 15" and "Fault 16") which are notoriously difficult to diagnose, the FDRs of the other fault types are all over 91%. Additionally, the t-SNE method is utilized to visualize the hierarchical learning feature process. Most data sample matrices of are clearly and correctly clustered by the DCNN in the t-SNE map. Then the model is tuned to reduce the false alarms. This brings out the trade-off between the needs of false alarm rate and fault diagnosis rate. Finally, the dynamic diagnostic performance and the diagnosis time are explored.

This method is prospective for industrial applications due to its outstanding fault diagnosis rates and false positive rates. However, since the model still relies on historical fault data samples, it is not applicable to diagnosis of faults without historical data or with little historical data. For a super complex chemical process which

generally has more than thousands of variables, input data dimension n will become a large number while the other dimension m will be relatively very small. Under this situation, how to design a DCNN architecture will become a serious problem. The future research work will be focused on designing a DCNN model for super complex chemical processes with thousands of process variables.

Acknowledgement

The authors gratefully acknowledge support from the National Natural Science Foundation of China (No. 61433001).

Reference

Ayubi Rad, M.A., Yazdanpanah, M.J., 2015. Designing supervised local neural network classifiers based on EM clustering for fault diagnosis of Tennessee Eastman process. Chemom. Intell. Lab. Syst. 146, 149–157. doi:10.1016/j.chemolab.2015.05.013.

Bathelt, A., Ricker, N.L., Jelali, M., 2015. Revision of the Tennessee eastman process model. IFAC PapersOnLine 309–314. doi:10.1016/j.ifacol.2015.08.199.

Bouvrie, J., 2006. Notes on convolutional neural networks. In Pract. 47–60. http://dx.doi.org/10.1016/j.protcy.2014.09.007.

Chiang, L.H., Russell, E.L., Braatz, R.D., 2000. Fault diagnosis in chemical processes using Fisher discriminant analysis, discriminant partial least squares, and principal component analysis. Chemom. Intell. Lab. Syst. 50, 243–252. doi:10.1016/S0169-7439(99)00061-1.

Chiang, L.H., Kotanchek, M.E., Kordon, A.K., 2004. Fault diagnosis based on Fisher discriminant analysis and support vector machines. Comput. Chem. Eng. 28, 1389–1401. doi:10.1016/j.compchemeng.2003.10.002.

Cho, J.-H., Lee, J.-M., Wook Choi, S., Lee, D., Lee, I.B., 2005. Fault identification for process monitoring using kernel principal component analysis. Chem. Eng. Sci. 60, 279–288. doi:10.1016/j.ces.2004.08.007.

Choi, S.W., Park, J.H., Lee, I.B., 2004. Process monitoring using a Gaussian mixture model via principal component analysis and discriminant analysis. Comput. Chem. Eng. 28, 1377–1387. doi:10.1016/j.compchemeng.2003.09.031.

Dai, Y., Zhao, J., 2011. Fault diagnosis of batch chemical processes using a dynamic time warping (DTW)-based artificial immune system. Ind. Eng. Chem. Res. 50, 4534–4544. doi:10.1021/ie101465b.

Downs, J.J., Vogel, E.F., 1993. A plant-wide industrial process control problem. Comput. Chem. Eng. 17, 245–255. doi:10.1016/0098-1354(93)80018-I.

Eslamloueyan, R., Shahrokhi, M., Bozorgmehri, R., 2003. Multiple simultaneous fault diagnosis via hierarchical and single artificial neural networks. Sci. Iran 10.

Eslamloueyan, R., 2011. Designing a hierarchical neural network based on fuzzy clustering for fault diagnosis of the Tennessee–Eastman process. Appl. Soft Comput. 11, 1407–1415. doi:10.1016/j.asoc.2010.04.012.

Fan, J.Y., Nikolaou, M., White, R.E., 1993. An approach to fault diagnosis of chemical processes via neural networks. AIChE J 39, 82–88. doi:10.1002/aic.690390109.

Fan, J., Wang, Y., 2014. Fault detection and diagnosis of non-linear non-Gaussian dynamic processes using kernel dynamic independent component analysis. Inf. Sci. 259, 369–379. doi:10.1016/j.ins.2013.06.021.

Ge, Z., Xie, L., Kruger, U., Song, Z., 2012. Local ICA for multivariate statistical fault diagnosis in systems with unknown signal and error distributions. AIChE J 58, 2357–2372. doi:10.1002/aic.12760.

Ghosh, K., Srinivasan, R., 2011. Immune-system-inspired approach to process monitoring and fault diagnosis. ind. eng. chem. Res 50, 1637–1651. doi:10.1021/ie100767c.

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. doi:10.1109/CVPR.2016.90.

He, Q.P., Wang, J., 2007. Fault detection using the k-nearest neighbor rule for semiconductor manufacturing processes. IEEE Trans. Semicond. Manuf. 20, 345–354. doi:10.1109/TSM.2007.907607.

Hinton, G.E., Roweis, S.T., 2002. Stochastic neighbor embedding. Adv. Neural Inf. Process. Syst. 833–840. http://books.nips.cc/papers/files/nips15/AA45.pdf.

Hinton, G.E., Salakhutdinov, R., 2006a. Reducing the dimensionality of data using neural networks. Science 313, 504–507 (80-.).

Hinton, G.E., Osindero, S., Teh, Y.W., 2006b. A fast learning algorithm for deep belief nets. Neural Comput 18, 1527–1554. doi:10.1162/neco.2006.18.7.1527.

Hinton, G.E., Srivastava, N., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.R., 2012. Improving neural networks by preventing co-adaptation of feature detectors. arXiv Prepr. arXiv:1207.0580.

Hsu, C.C., Chen, M.C., Chen, L.S., 2010. A novel process monitoring approach with dynamic independent component analysis. Control Eng. Pract. 18, 242–253. doi:10.1016/j.conengprac.2009.11.002.

Kano, M., Tanaka, S., Hasebe, S., Hashimoto, I., Ohno, H., 2003. Monitoring independent components for fault detection. AIChE J. 49, 969–976. doi:10.1002/aic.690490414.

Krizhevsky, A., Sutskever, I., Hinton, G.E., 2012. ImageNet classification with deep convolutional neural networks. Adv. Neural Inf. Process. Syst. 1–9. http://dx.doi.org/10.1016/j.protcy.2014.09.007.

Kulkarni, A., Jayaraman, V.K., Kulkarni, B.D., 2005. Knowledge incorporated support vector machines to detect faults in Tennessee Eastman process. Comput. Chem. Eng. 29, 2128–2133. doi:10.1016/j.compchemeng.2005.06.006.

LeCun, Y., Jackel, L.D., Boser, B., Denker, J.S., Graf, H.P., Guyon, I., Henderson, D., Howard, R.E., Hubbard, W., 1989. Handwritten digit recognition: applications of neural network chips and automatic learning. IEEE Commun. Mag. 27, 41–46. doi:10.1109/35.41400.

LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. Nature 521, 436–444. doi:10.1038/nature14539.

Lee, J.M., Qin, S.J., Lee, I.B., 2007. Fault detection of non-linear processes using kernel independent component analysis. Can. J. Chem. Eng. 85, 526–536.

Lin, M., Chen, Q., Yan, S., 2013. Network in network. arXiv Prepr. 10. doi:10.1109/ASRU.2015.7404828.

Lv, F., Wen, C., Bao, Z., Liu, M., 2016. Fault diagnosis based on deep learning. In: 2016 Am. Control Conf., pp. 6851–6856. doi:10.1109/ACC.2016.7526751.

Maaten, L.VanDer, Hinton, G., 2008. Visualizing data using t-SNE. J. Mach. Learn. Res. 1 (620), 267–284. doi:10.1007/s10479-011-0841-3.

Maurya, M.R., Rengaswamy, R., Venkatasubramanian, V., 2005. Fault diagnosis by qualitative trend analysis of the principal components. Chem. Eng. Res. Des. 83, 1122–1132. https://doi.org/10.1205/cherd.04280.

Maurya, M.R., Rengaswamy, R., Venkatasubramanian, V., 2007. A signed directed graph and qualitative trend analysis-based framework for incipient fault diagnosis. Chem. Eng. Res. Des. 85, 1407–1422. https://doi.org/10.1016/S0263-8762(07)73181-7.

Maurya, M.R., Paritosh, P.K., Rengaswamy, R., Venkatasubramanian, V., 2010. A framework for on-line trend extraction and fault diagnosis. Eng. Appl. Artif. Intell. 23, 950–960. https://doi.org/10.1016/j.engappai.2010.01.027.

MacGregor, J.F., Jaeckle, C., Kiparissides, C., Koutoudi, M., 1994. Process monitoring and diagnosis by multiblock PLS methods. AIChE J 40, 826–838. doi:10.1002/aic.690400509.

Mahadevan, S., Shah, S.L., 2009. Fault detection and diagnosis in process data using one-class support vector machines. J. Process Control 19, 1627–1639. doi:10.1016/j.jprocont.2009.07.011.

Plovoso, M.J., Kosanovich, K.A., 1994. Applications of multivariate statistical methods to process monitoring and controller design. Int. J. Control 59, 743–765. doi:10.1080/00207179408923103.

Rato, T., Reis, M., Schmitt, E., Hubert, M., De Ketelaere, B., 2016. A systematic comparison of PCA-based statistical process monitoring methods for high-dimensional, time-dependent processes. AIChE J. 62, 1478–1493. doi:10.1002/aic.15062.

Rato, T.J., Reis, M.S., 2017. Markovian and non-Markovian sensitivity enhancing transformations for process monitoring. Chem. Eng. Sci. 163, 223–233. https://doi.org/10.1016/j.ces.2017.01.047.

Russell, E.L., Chiang, L.H., Braatz, R.D., 2000. Fault detection in industrial processes using canonical variate analysis and dynamic principal component analysis. Chemom. Intell. Lab. Syst. 51, 81–93. doi:10.1016/S0169-7439(00)00058-7.

Shu, Y., Zhao, J., 2016. Fault diagnosis of chemical processes using artificial immune system with vaccine transplant. Ind. Eng. Chem. Res. 55, 3360–3371. doi:10.1021/acs.iecr.5b02646.

Simonyan, K., Zisserman, A., 2014. Very deep convolutional networks for large-scale image recognition. CoRR 1–14. http://arxiv.org/abs/1409.1556.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. J. Mach. Learn. Res. 15, 1929–1958. doi:10.1214/12-AOS1000.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., 2014. Going deeper with convolutions. arXiv Prepr. arXiv:1409.4842 1–9. doi:10.1109/CVPR.2015.7298594.

Venkatasubramanian, V., Chan, K., 1989. A neural network methodology for process fault diagnosis. AIChE J. 35, 1993–2002. doi:10.1002/aic.690351210.

Venkatasubramanian, V., Rengaswamy, R., Ka, S.N., Kavuri, S.N., Ka, S.N., 2003a. A review of process fault detection and diagnosis part II : qualitative models and search strategies. Comput. Chem. Eng. 27, 313–326. doi:10.1016/S0098-1354(02)00161-8.

Venkatasubramanian, V., Rengaswamy, R., Kavuri, S.N., Yin, K., 2003b. A review of process fault detection and diagnosis part III: process history based methods. Comput. Chem. Eng. doi:10.1016/S0098-1354(02)00162-X.

Venkatasubramanian, V., Rengaswamy, R., Kavuri, S.N., Yin, K., 2003c. A review of process fault detection and diagnosis part I: quantitative model based methods. Comput. Chem. Eng. 27, 293–311. doi:10.1016/S0098-1354(02)00160-6.

Watanabe, K., Matsuura, I., Abe, M., Kubota, M., Himmelblau, D.M., 1989. Incipient fault diagnosis of chemical processes via artificial neural networks. AIChE J. 35, 1803–1812. doi:10.1002/aic.690351106.

Watanabe, K., Hirota, S., Hou, L., Himmelblau, D.M., 1994. Diagnosis of multiple simultaneous fault via hierarchical artificial neural networks. AIChE J. 40, 839–848. doi:10.1002/aic.690400510.

Wise, B.M., Ricker, N.L., Veltkamp, D.F., Kowalski, B.R., 1990. Theoretical basis for the use of principal component models for monitoring multivariate processes. Process Control Qual. 1, 41–51.

Xie, D., Li, B., 2015. A hierarchical deep neural network for fault diagnosis on Tennessee–Eastman process. ICMLA, 2015 IEEE 14th International Conference doi:10.1109/ICMLA.2015.208.

Yélamos, I., Escudero, G., Graells, M., Puigjaner, L., 2009. Performance assessment of a novel fault diagnosis system based on support vector machines. Comput. Chem. Eng. 33, 244–255. doi:10.1016/j.compchemeng.2008.08.008.

Yin, S., Ding, S.X., Haghani, A., Hao, H., Zhang, P., 2012. A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee Eastman process. J. Process Control 22, 1567–1581. doi:10.1016/j.jprocont.2012.06.009.

Yu, J., Qin, S.J., 2008. Multimode process monitoring with bayesian inference-based finite Gaussian mixture models. AIChE J. 54, 1811–1829. doi:10.1002/aic.11515.

Zhang, Y.W., 2008. Fault detection and diagnosis of nonlinear processes using improved kernel independent component analysis (KICA) and support vector machine (SVM). Ind. Eng. Chem. Res. 47, 6961–6971. doi:10.1021/Ie071496x.

Zhang, Y., Hu, Z., 2011. Multivariate process monitoring and analysis based on multiscale KPLS. Chem. Eng. Res. Des. 89, 2667–2678. doi:10.1016/j.cherd.2011.05.005.

Zhang, Z., Zhao, J., 2017. A deep belief network based fault diagnosis model for complex chemical processes. Comput. Chem. Eng. doi:10.1016/j.compchemeng.2017.02.041.

Zhu, Z.B., Song, Z.H., 2011. A novel fault diagnosis system using pattern classification on kernel FDA subspace. Expert Syst. Appl. 38, 6895–6905. doi:10.1016/j.eswa.2010.12.034.